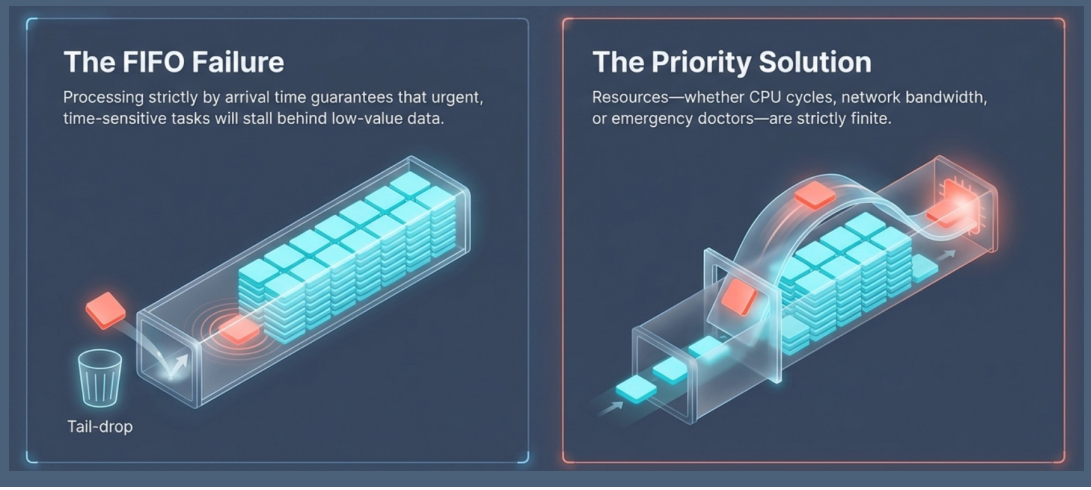
Computer Olympiad Camp 2

Priority Queue and Heap and Some STL libraries

Thapanapong Rukkanchanunt [30 August 2024]. Adapted for CompO by Kamonphop Srisopha

Introduction to Priority Queue

- A stack is last in, first out (LIFO)
- A queue is first in, first out (FIFO)
- A priority queue is least-first-out or most-first-out
  - The most “important” element is the first one removed
  - e.g., ผู้มีความจำเป็นด้านร่างกาย อาจได้เข้ารับบริการก่อน โดยไม่ต้องต่อคิว



Applications of Priority Queues

- Priority queues ถูกใช้ในงานที่จำเป็นต้องจัดสรรลำดับก่อนหลังในการเข้าถึง resource (memory, cpu)
  จึงเป็น concept ที่จำเป็นมากใน computer science
- Example of applications:
  - การจัดลำดับ task ในระบบปฏิบัติการ
  - การทดแทนเพจ (Page replacement) ในระบบจัดการหน่วยความจำ

Example: Task Scheduling in OS

- ต้องการเลือกว่า task (process) ไหนควรจะต้องถูก execute ด้วยลำดับอย่างไร
- เป้าหมายคือ
  - CPU utilization: CPU ต้องไม่ว่าง
  - Throughput: จำนวน task ที่ execute ได้ต่อหน่วยเวลา
  - Turnaround Time: เวลาตั้งแต่ submit process จนกระทั่ง process execute เสร็จ
- Priority Scheduling: process แต่ละตัวจะมี priority อยู่ และเราเลือกที่จะ execute process ที่มี priority
  สูงสุดก่อน

Scenario

- Suppose we have 4 tasks. Each task has arrival time, burst time (time needed for execution) and priority value (lower = more important).

Task	Arrival	Burst	Priority
T1	0	5	3
T2	1	4	1
T3	2	2	4
T4	3	1	2

At time = 0

Task	Arrival	Burst	Priority
T1	0	5	3
T2	1	4	1
T3	2	2	4
T4	3	1	2

- Task T1 has arrived. Our CPU is free so we will do T1 for 5 unit of time.

Time	1	2	3	4	5
Task	T1	T1	T1	T1	T1

At time = 5

Task	Arrival	Burst	Priority
T1	0	5	3
T2	1	4	1
T3	2	2	4
T4	3	1	2

- Task T1 is completed. During the time, T2, T3, and T4 arrived. The next task we will pick is one with highest priority (lowest priority value). We will do T2 for 4 unit of time.

Time	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
Task	T1	T1	T1	T1	T1	T2	T2	T2	T2

At time = 9

Task	Arrival	Burst	Priority
T1	0	5	3
T2	1	4	1
T3	2	2	4
T4	3	1	2

- Task T2 is completed. That means that T3 and T4 are in the task queue. The next task we will pick is one with highest priority (lowest priority value). We will do T4 for 1unit of time.

Time	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Task	T1	T1	T1	T1	T1	T2	T2	T2	T2	T4

At time = 10

Task	Arrival	Burst	Priority
T1	0	5	3
T2	1	4	1
T3	2	2	4
T4	3	1	2

- Task T4 is completed. T3 is still in the task queue. We will do T3 for 2 unit of time.

Time	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
Task	T1	T1	T1	T1	T1	T2	T2	T2	T2	T4	T3	T3

Turnaround Time

The total delay a task experiences before completion, which reflects how efficiently the OS finishes work.

Task	Arrival	Burst	Priority
T1	0	5	3
T2	1	4	1
T3	2	2	4
T4	3	1	2

- To compute the turnaround time for each task, we subtract arrival time from its completion time.
  - T1: TAT = 5 - 0 = 5
  - T2: TAT = 9 - 1 = 8
  - T3: TAT = 12 - 2 = 10
  - T4: TAT = 10 - 3 = 7

Time	1	2	3	4	5	6	7	8	9	10	11	12
Task	T1	T1	T1	T1	T1	T2	T2	T2	T2	T4	T3	T3

Example: Memory Management

- Imagine your computer is running lots of programs at the same time (like a game, a browser, and music). Each program needs space in memory (RAM) to work.
- But RAM is limited; it can't hold everything at once.
- So the system uses **virtual memory** to extend RAM using disk space
- Programs are split into small pieces called **pages**
  - When the RAM is full, the operating system must decide which pages to swap out to disk (also known as "**paging out**") to free up space for new pages.
- Each page in memory can be assigned a priority based on certain criteria:
  - Frequency of access, recency of access, process priority

Scenario

- Suppose our system is using **a least recently used (LRU) policy** for page swap.
  - i.e., Removes the page that was **used the longest time ago**.
  - The system can only **hold 3 pages** in RAM at a time.

RAM	FREE	FREE	FREE
Last Access			

The following is the order of page to be assessed:

P1	P2	P3	P4	P5	P3	P1

First 3 Pages

P1	P2	P3	P4	P5	P3	P1

- The first 3 pages can fit in to our memory so we keep them all.
- We also need to store the time when they are accessed.

RAM	P1	P2	P3
Last Access	1	2	3

4th Page

P1	P2	P3	P4	P5	P3	P1

- Memory is full so we need to “page out” a page.
- P1 was the least recently accessed so P4 replaces P1.

RAM	P4	P2	P3
Last Access	4	2	3

5th Page

P1	P2	P3	P4	P5	P3	P1

- Memory is full so we need to “page out” a page.
- P2 was the least recently accessed so P5 replaces P2.

RAM	P4	P5	P3
Last Access	4	5	3

6th Page

P1	P2	P3	P4	P5	P3	P1

- P3 is already in memory so we don’t need to page out any pages.
- However, we must update last access for P3.

RAM	P4	P5	P3
Last Access	4	5	6

Last Page

P1	P2	P3	P4	P5	P3	P1

- Memory is full so we need to “page out” a page.
- P4 was the least recently accessed so P1 replaces P4.

RAM	P1	P5	P3
Last Access	7	5	6

So, Priority Queue is Needed

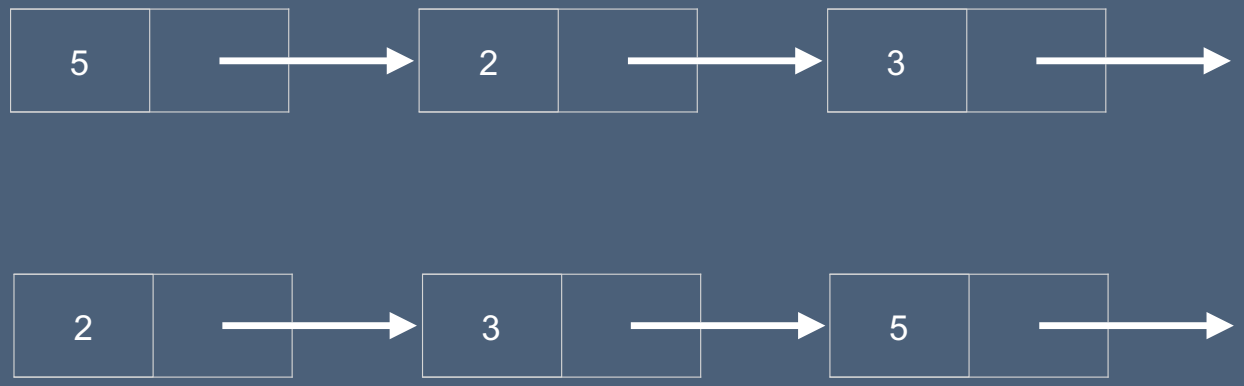
But what data structure should we use for this?

Priority Queue Implementation with A Simple Array (List)

Exercise 1

- Implement priority queue using a simple list

Priority Queue Implementation with Linked List



Exercise 2

- Implement priority queue using a linked list

So, What are the problems with both data structures for priotity queue?

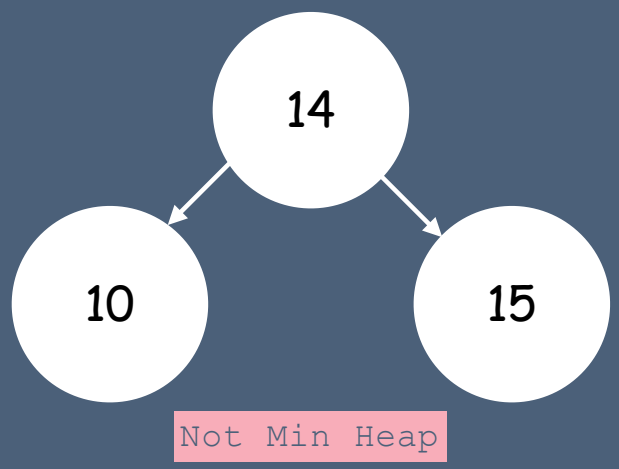
Priority Queue Implementation with List or Linked List

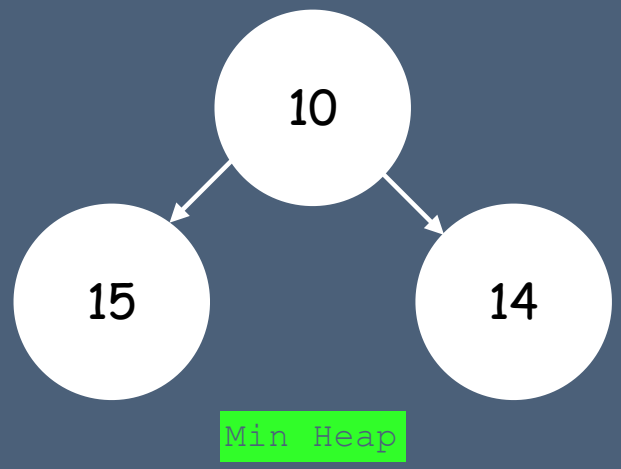
- Unsorted List
  - Insertion is fast (insert at the end of the list/linked list.) O(1)
  - Remove min is very slow (traverse the list/linked list). + Extra work for a simple list when removing an element (Shifting) – O(n)
- Sorted List
  - Insertion is very slow because you have to find the right place (+extra work for shifting element for a simple list) . O(n)
  - Remove min is fast (e.g., at the end – keep smallest at the end, no shifting needed) - O(1)

Another Possible Structure

- Heap is a binary tree (คนละ concept กับ Heap เรื่อง Memory)
  - For each node, its priority value is lower than its children's.

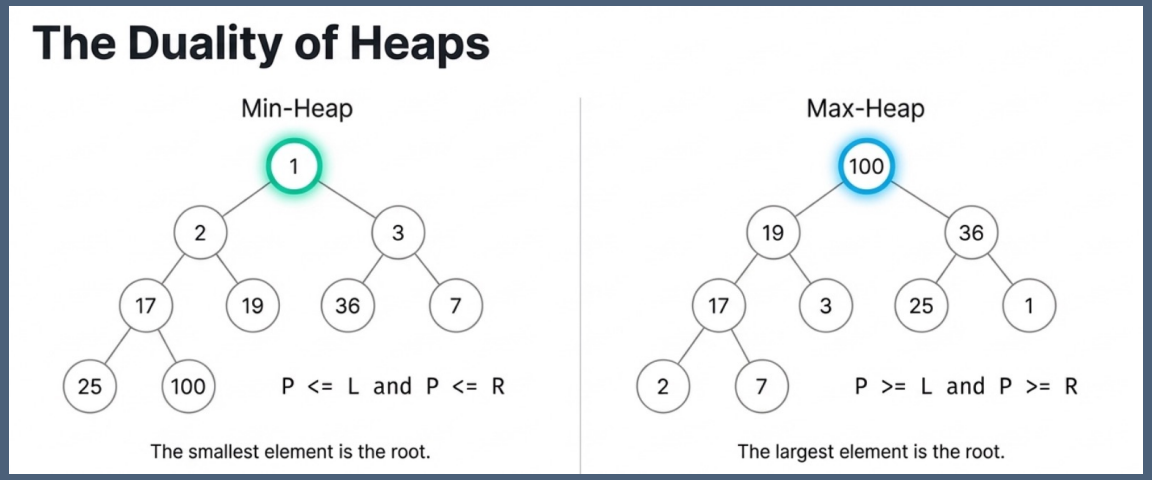
P <= L, P <= R

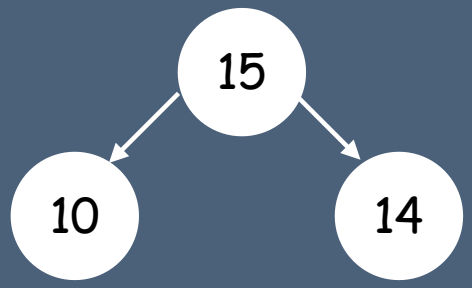




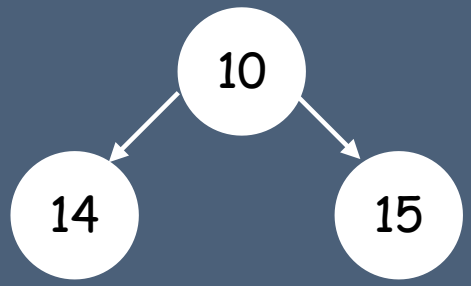
Types of Heaps

- Min heap
  - For each node, its priority value is lower than its children's.
- Max heap
  - For each node, its priority value is higher than its children's.





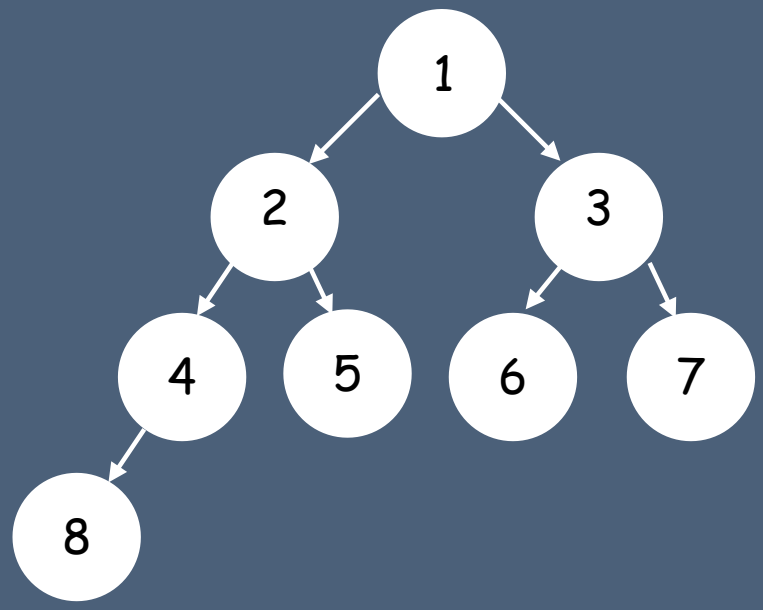
Max Heap



Min Heap

Heap Structure

- Heap is a complete binary tree (what does that mean?)

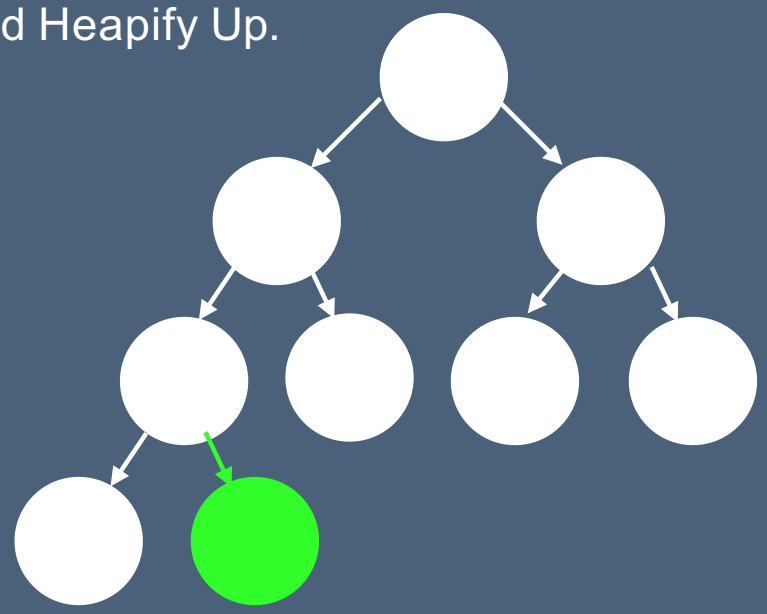


Heap Operations

- Just like other data structures we learn, a (min) heap has common operations:
  - **Insert(Value)**
  - **Remove Min (i.e., pop min/delete)**
  - **Retrieve Min**
- The important detail is that **after insert and delete**, the constraint on each node must be maintained.
  - There will be procedure to ensure this always holds.

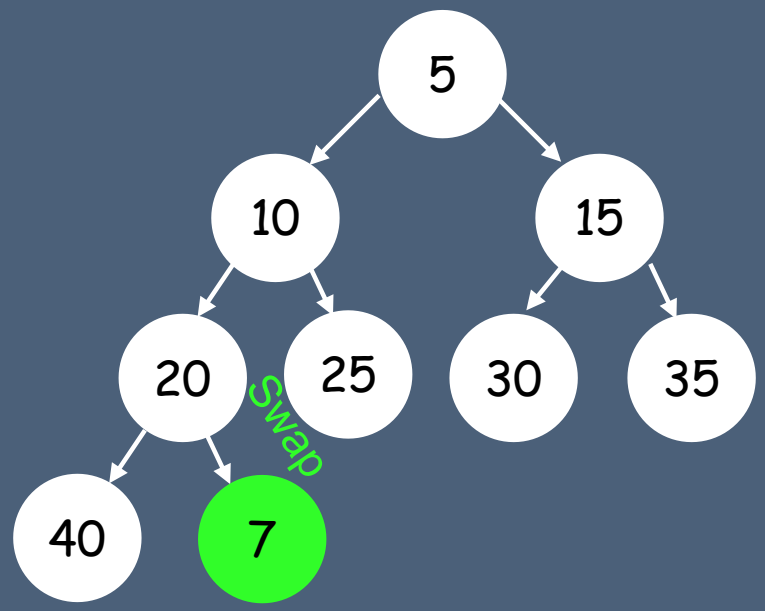
Insert

- We always insert at the next available location in order-level manner.
- Starting from that node, we will fix the structure from the bottom up.
  - This procedure is called Heapify Up.



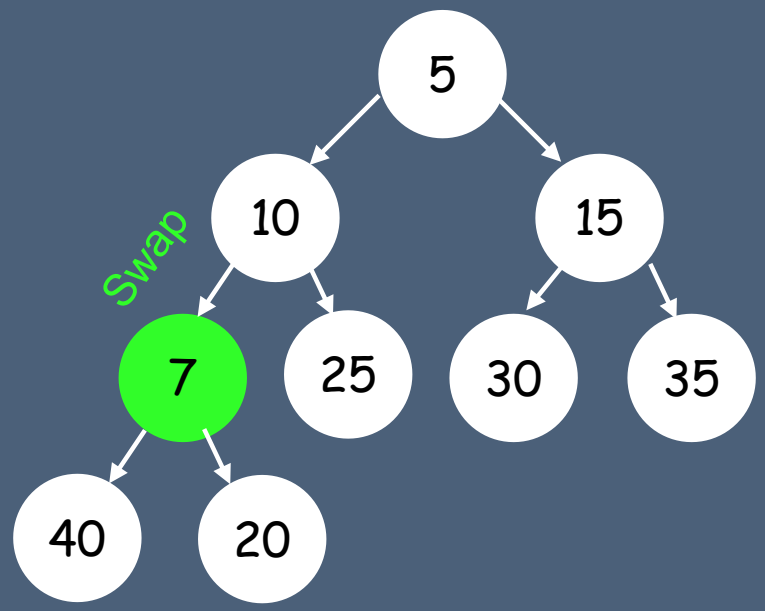
Heapify Up

- If parent has higher priority value, swap it.
- Repeat until you reach the root.



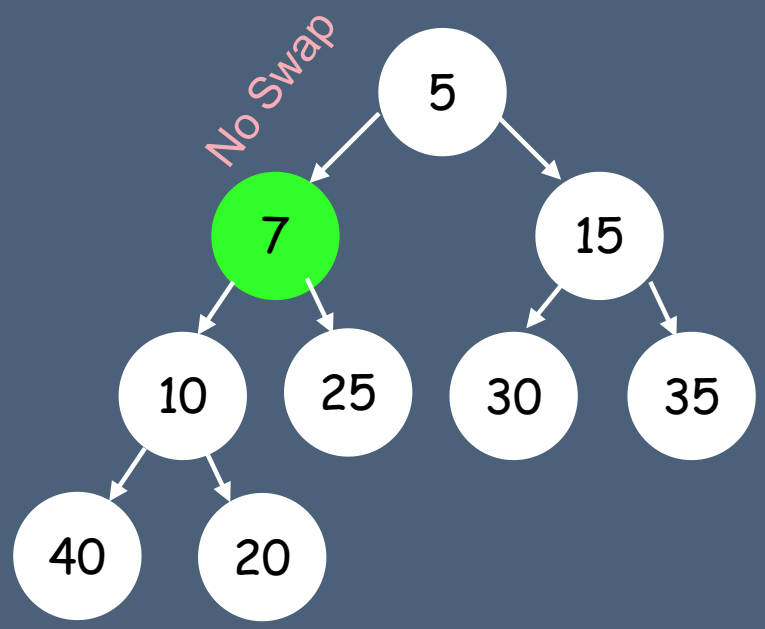
Heapify Up

- If parent has higher priority value, swap it.
- Repeat until you reach the root.



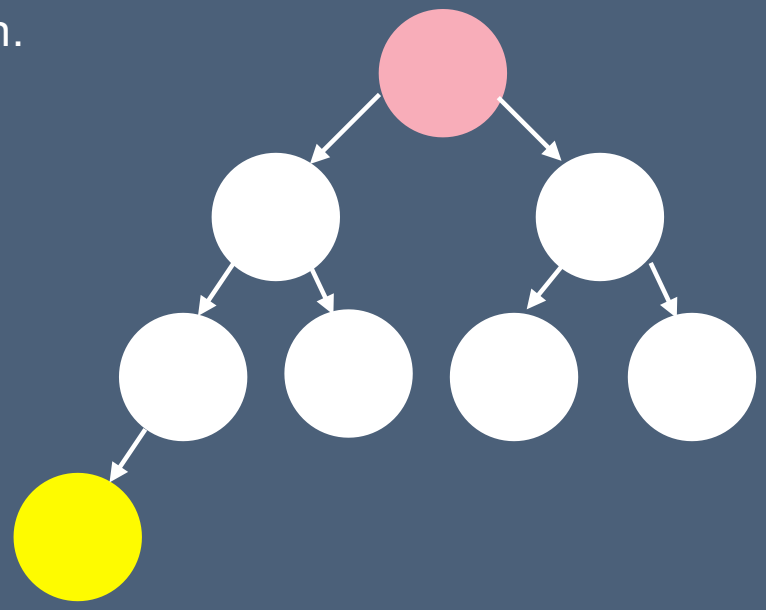
Heapify Up

- If parent has higher priority value, swap it.
- Repeat until you reach the root.



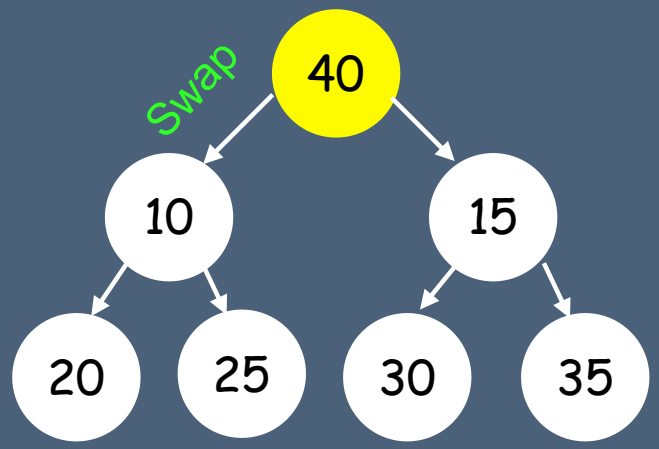
Remove Min

- **We always remove at the root.**
- For a successor, we pick the last location in the level-order manner.
- From the root, we will fix the structure from top down.
  - This procedure is called Heapify Down.



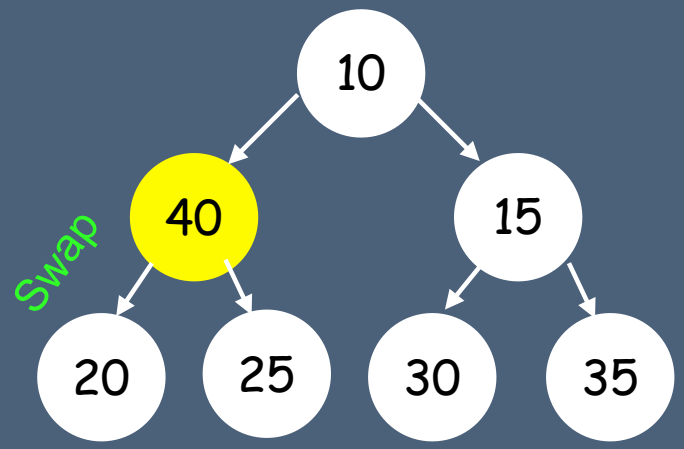
Heapify Down

- If **the lowest priority value** is at left or right child, we swap node with that child.
- Repeat the same process on the swapped child **until we hit the leaf.**



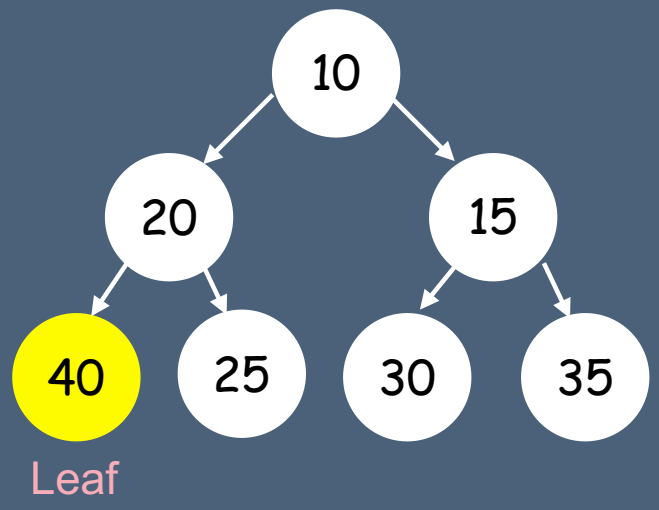
Heapify Down

- If the lowest priority value is at left or right child, we swap node with that child.
- Repeat the same process on the swapped child until we hit the leaf.



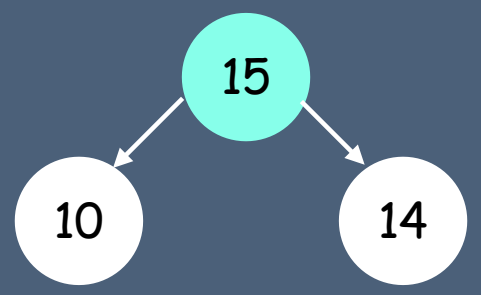
Heapify Down

- If the lowest priority value is at left or right child, we swap node with that child.
- Repeat the same process on the swapped child until we hit the leaf.

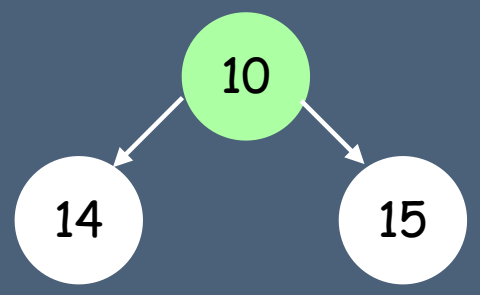


Recap

- Min heap
  - For each node, its priority value is lower than its children's.
- Max heap
  - For each node, its priority value is higher than its children's.



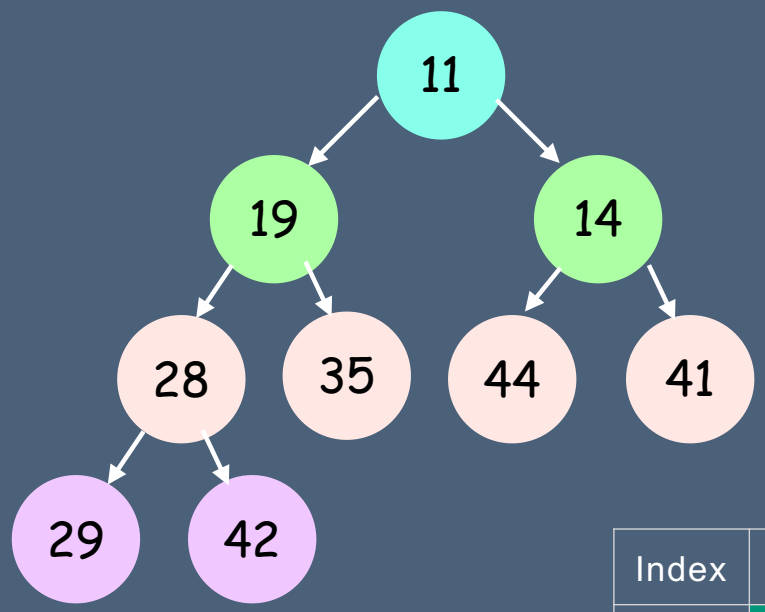
Max Heap

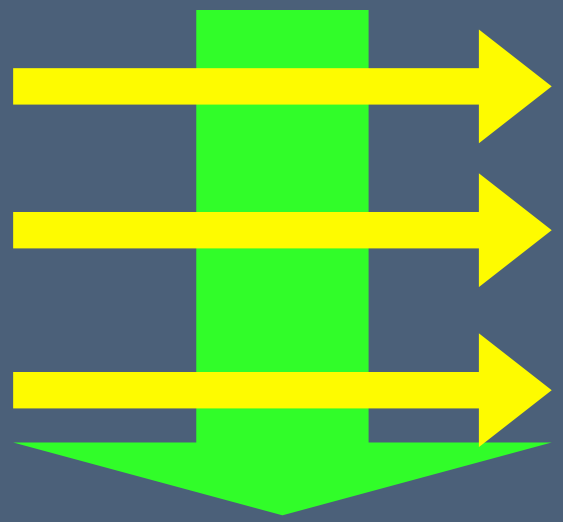


Min Heap

Heap in Array

- Index of the array corresponds to the level-order traversal of the tree.





Index	0	1	2	3	4	5	6	7	8
Value	11	19	14	28	35	44	41	29	42

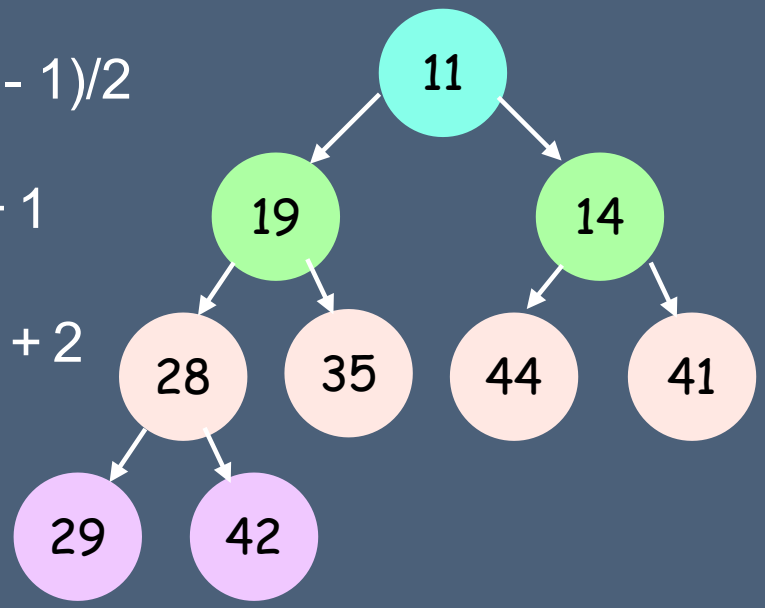
Heap Class

- Because we use vector, the heap class can be simple:

```
class Heap {
private:
    std::vector<int> heap;
```

Parent-Child Relationship

- If we know the index of current node, can we calculate the index of parent/child node?
  - Index of parent node = (index of current node - 1)/2
  - Index of left child = 2 * index of current node + 1
  - Index of right child = 2 * index of current node + 2



Index	0	1	2	3	4	5	6	7	8
Value	11	19	14	28	35	44	41	29	42

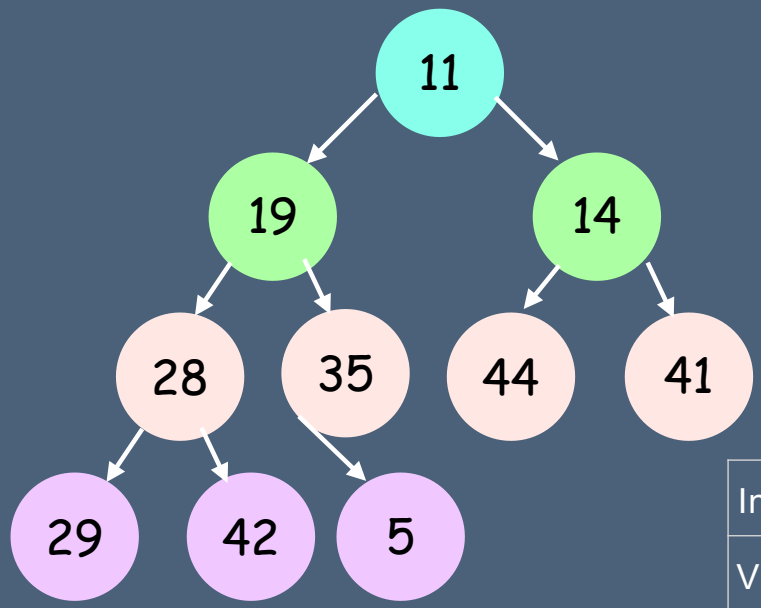
Parent-Child Relationship

- If we know the index of current node, can we calculate the index of parent/child node?
  - Index of parent node = (index of current node - 1)/2
  - Index of left child = 2 * index of current node + 1
  - Index of right child = 2 * index of current node + 2

```
int parent(int index) { return (index - 1) / 2; }
int leftChild(int index) { return (2 * index + 1); }
int rightChild(int index) { return (2 * index + 2); }
```

Insertion

- Inserting in heap is equivalent to inserting at the last available location in the array.
- Then, do heapify up procedure.



Index	0	1	2	3	4	5	6	7	8	9
Value	11	19	14	28	35	44	41	29	42	5

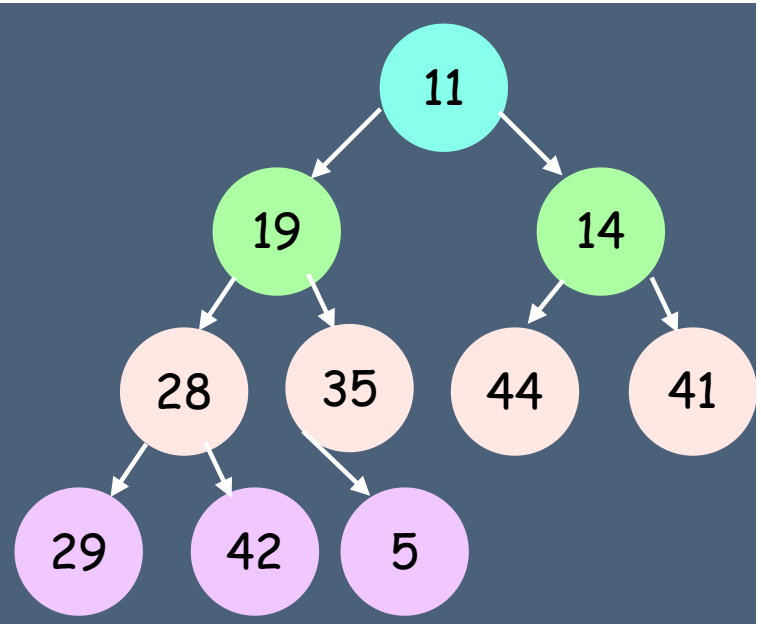
Insertion Code

- Inserting in heap is equivalent to inserting at the last available location in the array.
- Then, do heapify up procedure.

```
void insert(int key) {
    heap.push_back(key);
    int index = heap.size() - 1;
    heapifyUp(index);
}
```

Location of last element

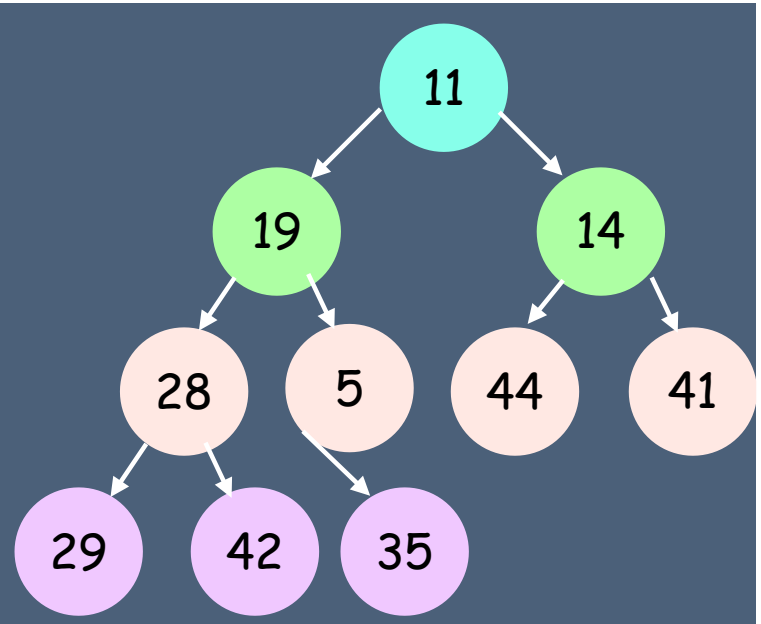
Heapify Up



- My index = i
- My parent index = (i-1)/2
- If value at my index is less than the value of my parent, I swap.
- Repeat the process until root.

Index	0	1	2	3	4	5	6	7	8	9
Value	11	19	14	28	35	44	41	29	42	5

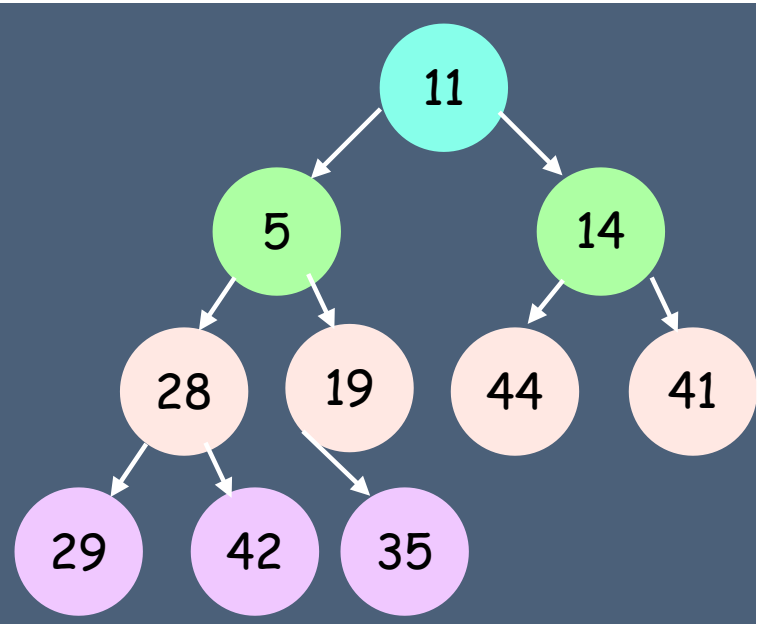
Heapify Up



- My index = i
- My parent index = (i-1)/2
- If value at my index is less than the value of my parent, I swap.
- Repeat the process until root.

Index	0	1	2	3	4	5	6	7	8	9
Value	11	19	14	28	5	44	41	29	42	35

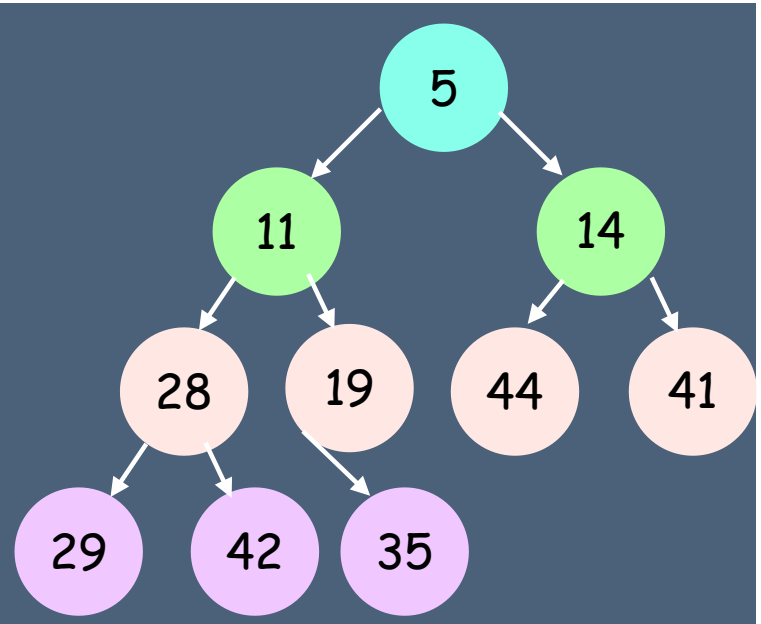
Heapify Up



- My index = i
- My parent index = (i-1)/2
- If value at my index is less than the value of my parent, I swap.
- Repeat the process until root.

Index	0	1	2	3	4	5	6	7	8	9
Value	11	5	14	28	19	44	41	29	42	35

Heapify Up



- My index = i
- My parent index = (i-1)/2
- If value at my index is less than the value of my parent, I swap.
- Repeat the process until root.

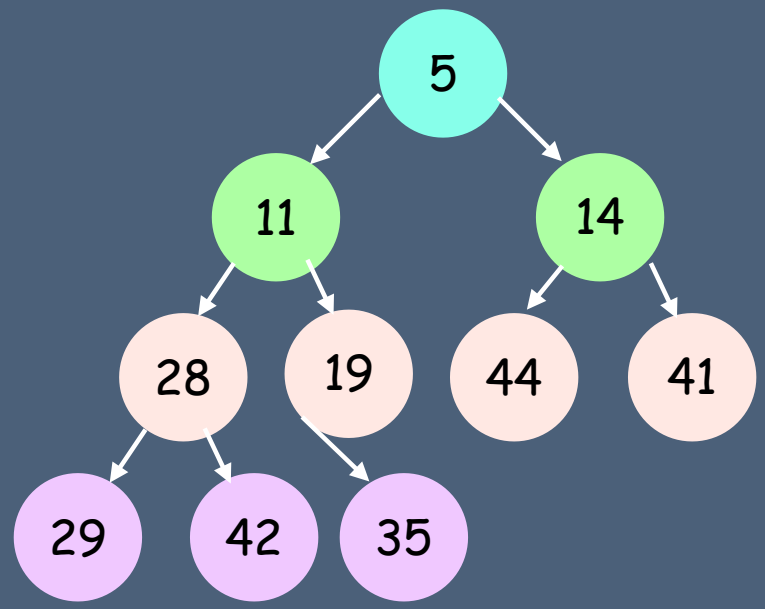
Index	0	1	2	3	4	5	6	7	8	9
Value	5	11	14	28	19	44	41	29	42	35

Heapify Up Code

- My index = i
- My parent index = (i-1)/2
- If value at my index is less than the value of my parent, I swap.
- Repeat the process until root.

```
void heapifyUp(int index) {
    if (index && heap[parent(index)] > heap[index]) {
        std::swap(heap[index], heap[parent(index)]);
        heapifyUp(parent(index));
    }
}
```

Remove Min

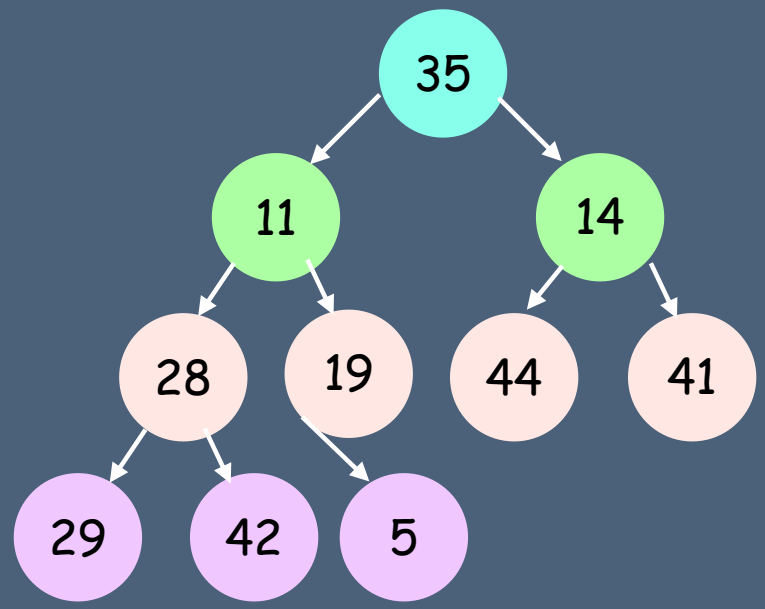


ret
5

- Save the root value to a variable.
- Swap the last position in the array with root.
- Remove last element of the array.
- Then do heapify down procedure.
- Return the saved value.

Index	0	1	2	3	4	5	6	7	8	9
Value	5	11	14	28	19	44	41	29	42	35

Remove Min

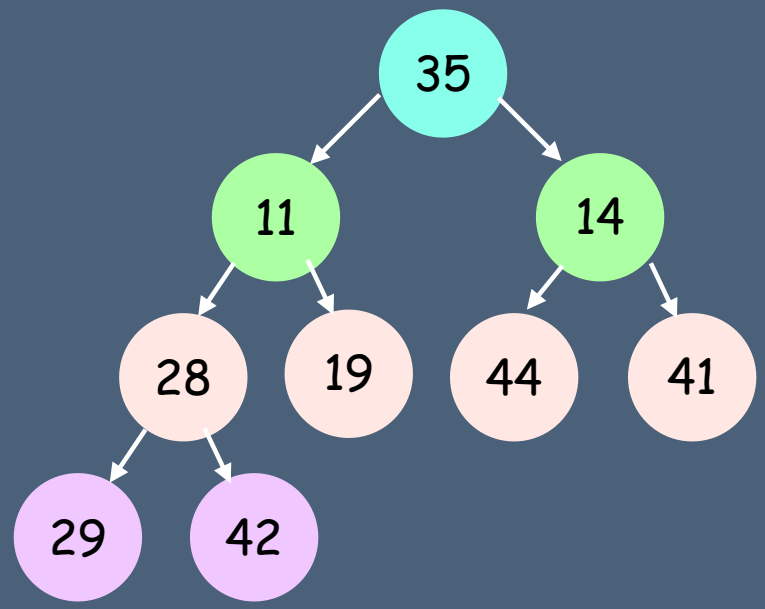


ret
5

- Save the root value to a variable.
- Swap the last position in the array with root.
- Remove last element of the array.
- Then do heapify down procedure.
- Return the saved value.

Index	0	1	2	3	4	5	6	7	8	9
Value	35	11	14	28	19	44	41	29	42	5

Remove Min

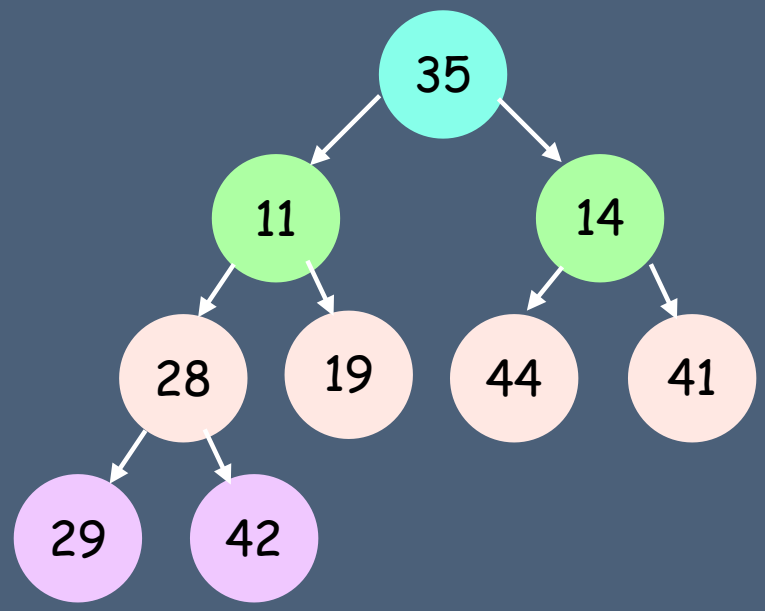


ret
5

- Save the root value to a variable.
- Swap the last position in the array with root.
- Remove last element of the array.
- Then do heapify down procedure.
- Return the saved value.

Index	0	1	2	3	4	5	6	7	8
Value	35	11	14	28	19	44	41	29	42

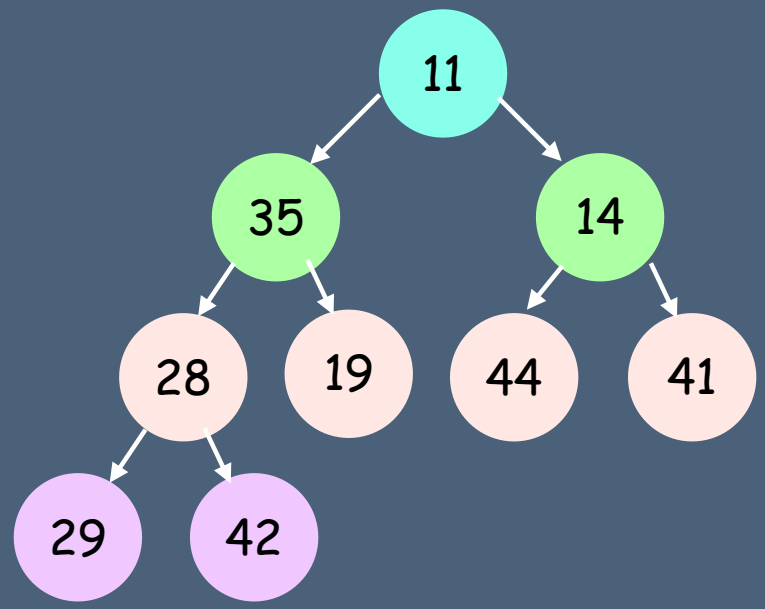
Heapify Down



- My index = i
- Left child = l = 2 * i + 1
- Right child = r = 2 * i + 2
- Find the minimum among i, l and r.
  - Swap i with the minimum.
- Repeat until leaf.

Index	0	1	2	3	4	5	6	7	8
Value	35	11	14	28	19	44	41	29	42

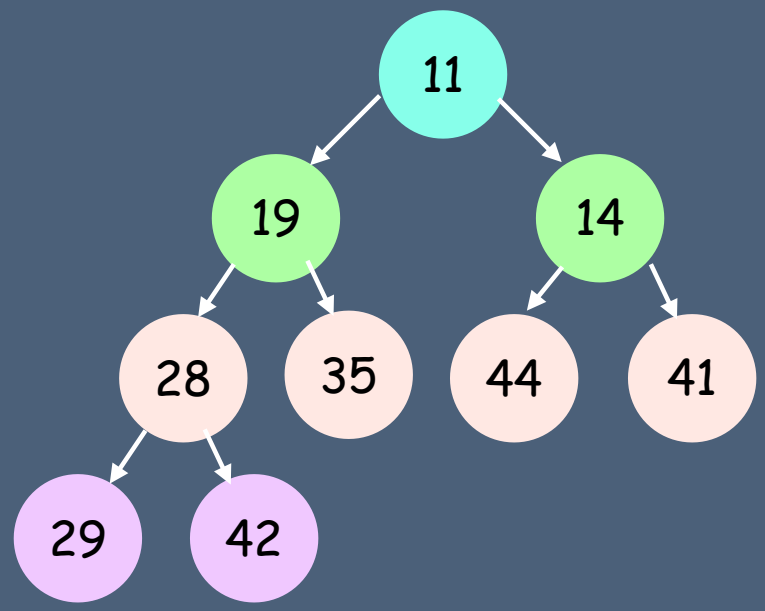
Heapify Down



- My index = i
- Left child = l = 2 * i + 1
- Right child = r = 2 * i + 2
- Find the minimum among i, l and r.
  - Swap i with the minimum.
- Repeat until leaf.

Index	0	1	2	3	4	5	6	7	8
Value	11	35	14	28	19	44	41	29	42

Heapify Down

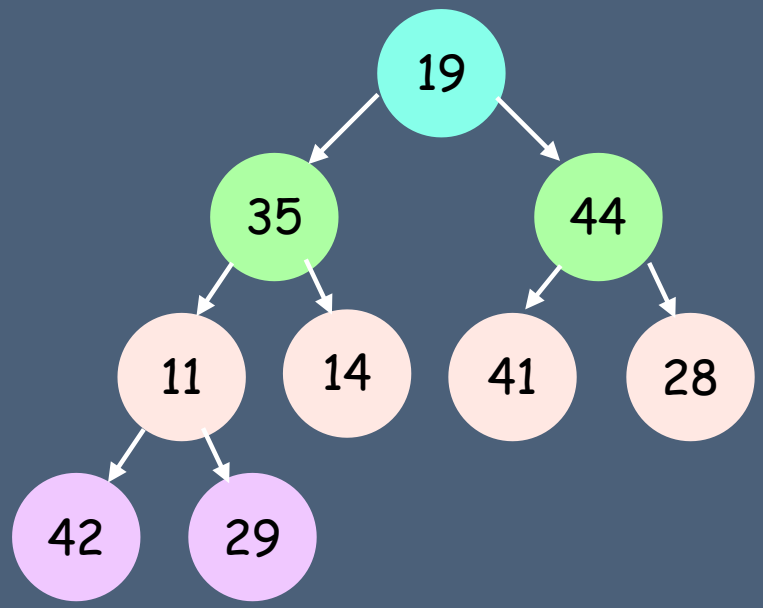


- My index = i
- Left child = l = 2 * i + 1
- Right child = r = 2 * i + 2
- Find the minimum among i, l and r.
  - Swap i with the minimum.
- Repeat until leaf.

Index	0	1	2	3	4	5	6	7	8
Value	11	19	14	28	35	44	41	29	42

Build Heap

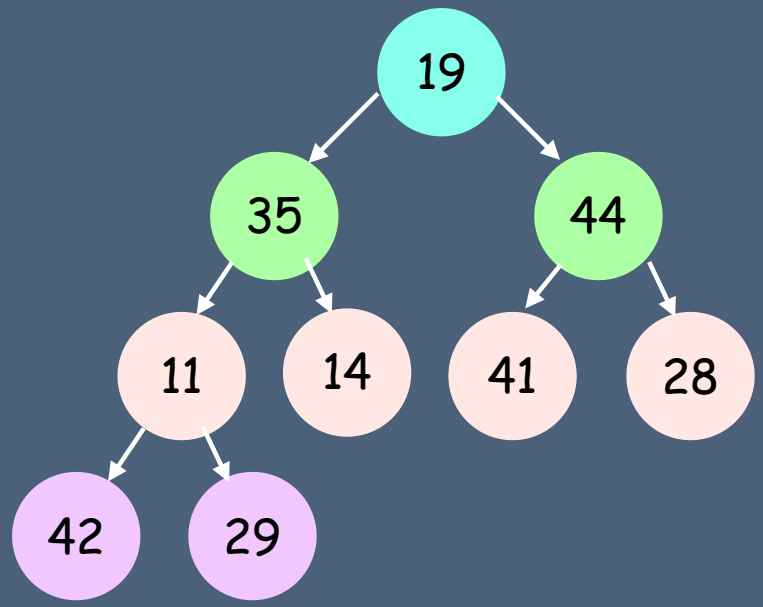
- We can make any array to heap array.
- Simple algorithm
  - Start from last non-leaf node (n//2-1)
  - For each index i down to 0, do heapify down at that position.



Index	0	1	2	3	4	5	6	7	8
Value	19	35	44	11	14	41	28	42	29

Build Heap

- We can make any array to heap array.
- Simple algorithm
  - Start from last non-leaf node (n/2-1)
  - For each index i down to 0, do heapify down at that position.

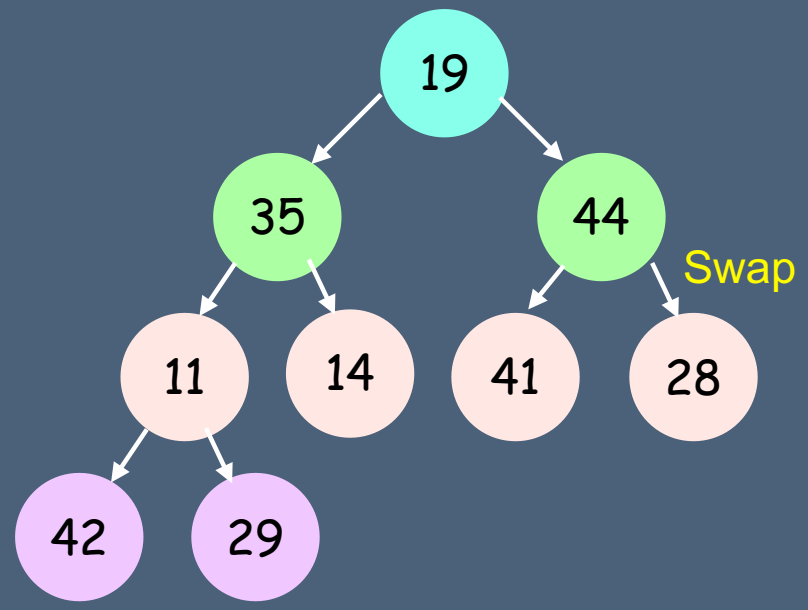


Start = 9//2 – 1 = 3
Heapify at indices 3, 2, 1, 0

Index	0	1	2	3	4	5	6	7	8
Value	19	35	44	11	14	41	28	42	29

Build Heap

- We can make any array to heap array.
- Simple algorithm
  - Start from last non-leaf node (n/2-1)
  - For each index i down to 0, do heapify down at that position.

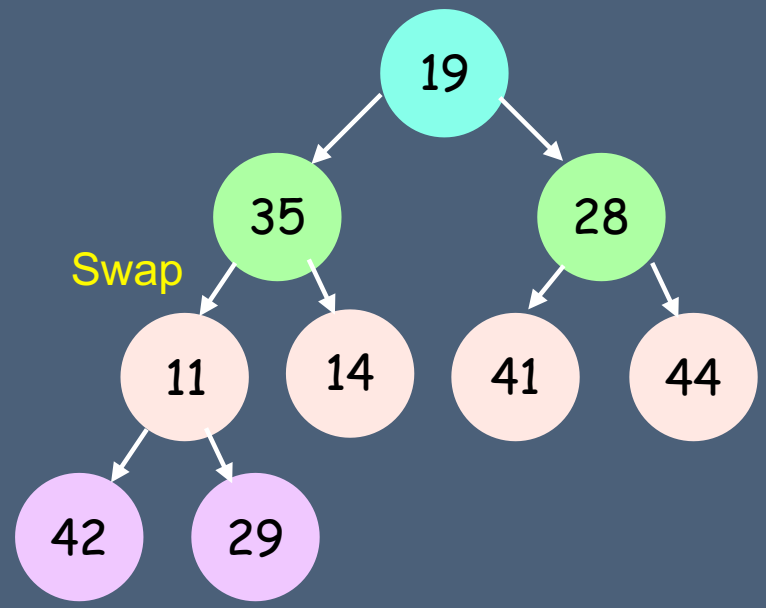


Next index = 2

Index	0	1	2	3	4	5	6	7	8
Value	19	35	44	11	14	41	28	42	29

Build Heap

- We can make any array to heap array.
- Simple algorithm
  - Start from last non-leaf node (n//2-1)
  - For each index i down to 0, do heapify down at that position.

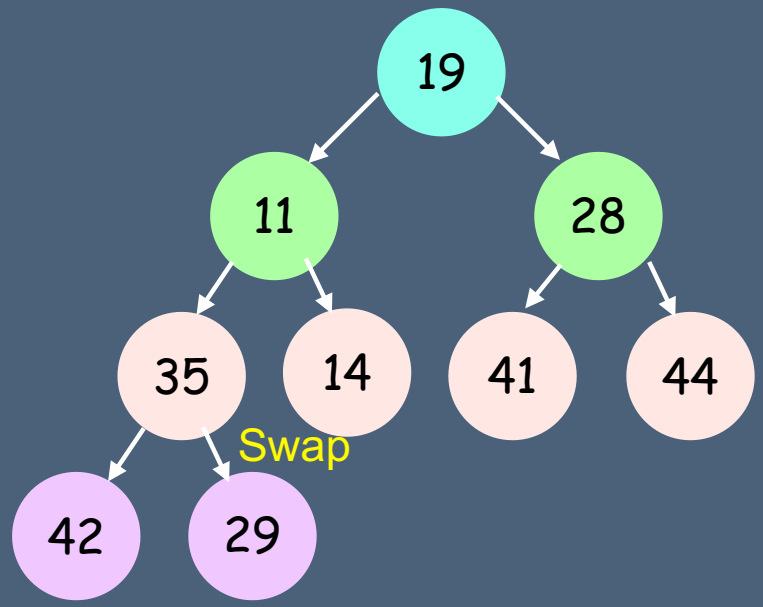


Next index = 1

Index	0	1	2	3	4	5	6	7	8
Value	19	35	28	11	14	41	44	42	29

Build Heap

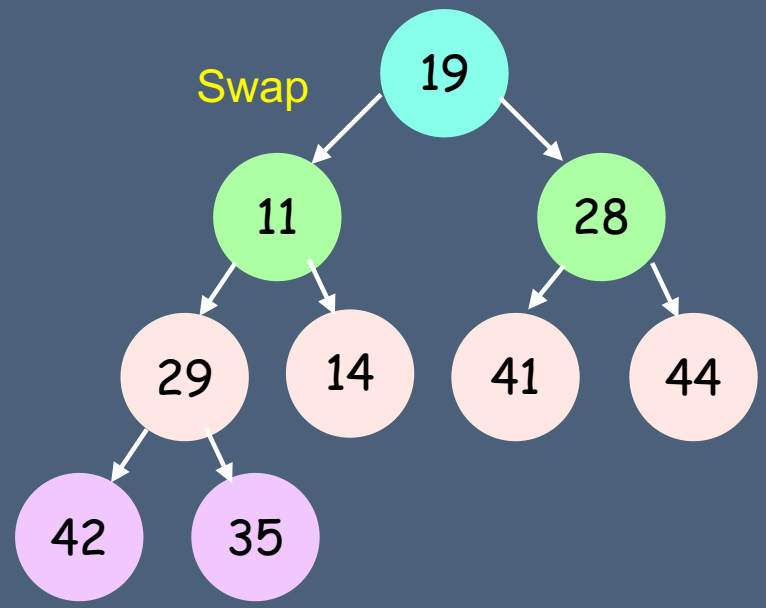
- We can make any array to heap array.
- Simple algorithm
  - Start from last non-leaf node (n/2-1)
  - For each index i down to 0, do heapify down at that position.



Index	0	1	2	3	4	5	6	7	8
Value	19	11	28	35	14	41	44	42	29

Build Heap

- We can make any array to heap array.
- Simple algorithm
  - Start from last non-leaf node (n//2-1)
  - For each index i down to 0, do heapify down at that position.

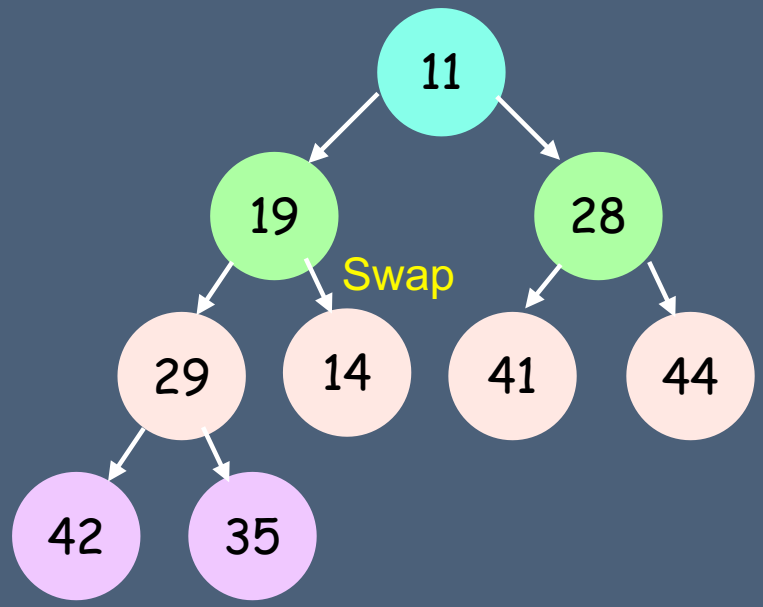


Next index = 0

Index	0	1	2	3	4	5	6	7	8
Value	19	11	28	29	14	41	44	42	35

Build Heap

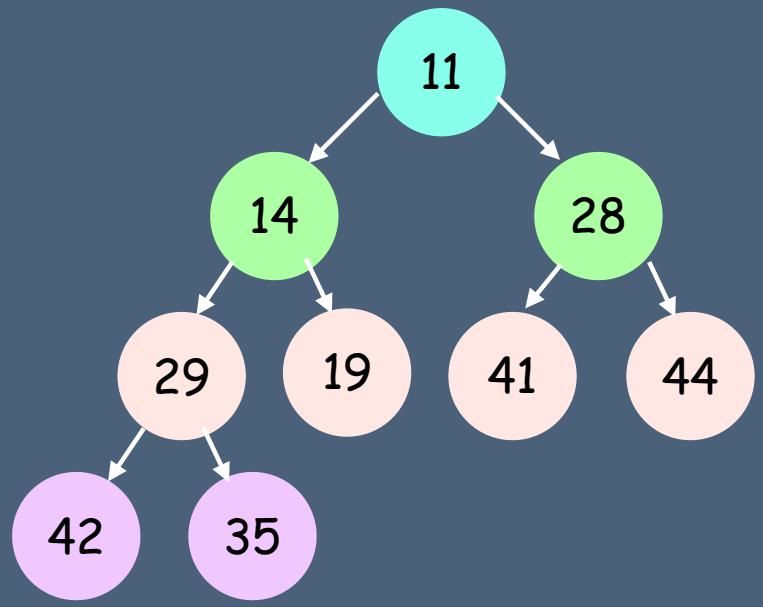
- We can make any array to heap array.
- Simple algorithm
  - Start from last non-leaf node (n/2-1)
  - For each index i down to 0, do heapify down at that position.



Index	0	1	2	3	4	5	6	7	8
Value	11	19	28	29	14	41	44	42	35

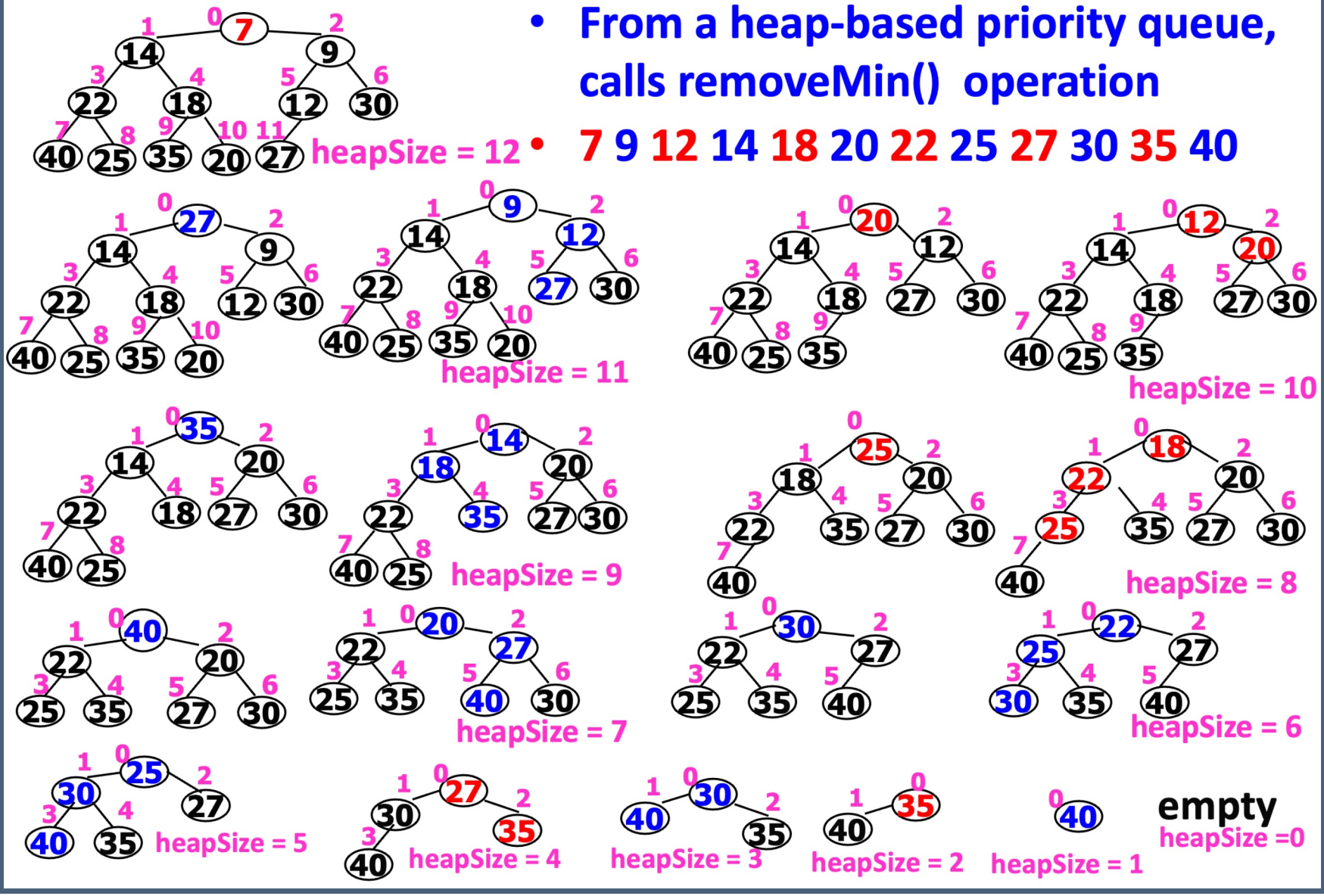
Build Heap

- We can make any array to heap array.
- Simple algorithm
  - Start from last non-leaf node (n//2-1)
  - For each index i down to 0, do heapify down at that position.



Index	0	1	2	3	4	5	6	7	8
Value	11	14	28	29	19	41	44	42	35

- From a heap-based priority queue, calls removeMin() operation
- 7 9 12 14 18 20 22 25 27 30 35 40



Exercise 3

- Implement priority queue using a heap
- (Complete the implementation)

Time Complexity?

- Insert
- Remove Min
- Get Min
- Build Heap

Build Heap

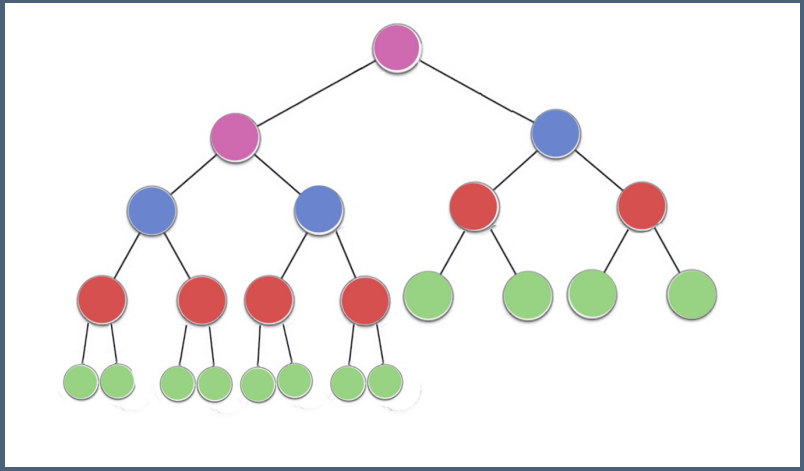
n/2^1 with height 0 – do 0 work

n/2^2 with height 1 – do at most 1 work (1 swap) each

n/2^3 with height 2 – do at most 2 works (2 swaps) each

n/2^4 with height 3 – do at most 3 works (3 swaps) each

n/2^5 with height 4 – do at most 4 works (4 swaps) each



Total work =

(n/2^1 * 0) + (n/2^2 * 1) + (n/2^3 * 2) + … + (n/(2^(h+1)) * h)

Generalize = n * (1/4 + 2/8 + 3/16 + … + h/2^(h+1)) = **O(n)**

Heap in STL

Time Complexity: O(N)	**where N is the number of elements.**
Auxiliary Space: O(1)	since no extra space is used.

- make_heap()

```
std::make_heap(begin_iterator, end_iterator);
```

- Used to convert the given range in a container to a heap

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {19, 35, 44, 11, 14, 41, 28};

    // Build heap
    make_heap(v.begin(), v.end());

    cout << "Max element: " << v.front() << endl;

    return 0;
}
```

Note: Default is MAX-heap

For min heap

- Define custom comparator
- Pass it to make_heap()
- Or pass std::greater<int>()

```cpp
#include<iostream>
#include<algorithm>
#include<vector>
using namespace std;

// comparator function to make min heap
struct greaters{
    bool operator()(const long& a,const long& b) const{
        return a>b;
    }
};

int main() {
    vector<int> v = { 15, 6, 7, 9, 11, 45 };

    // using make_heap() to transform vector into
    // a min heap
    make_heap(v.begin(),v.end(), greaters());

    // checking if heap using
    // front() function
    cout << "The minimum element of heap is : ";
    cout << v.front();
}
```

or with #include <functional>

```cpp
#include <functional>
using namespace std;

int main() {
    vector<int> v = {19, 35, 44, 11, 14, 41, 28};

    // Create min-heap
    make_heap(v.begin(), v.end(), greater<int>());

    // Smallest element
    cout << "Min element: " << v.front() << endl;

    // Remove elements one by one
    while (!v.empty()) {
        pop_heap(v.begin(), v.end(), greater<int>());
        cout << v.back() << " ";
        v.pop_back();
    }

    return 0;
}
```

Push heap (max heap)

- ใช้จัดเรียง heap ใหม่หลังจาก push_back()
- This version is for max heap
- Use custom comparator for min heap

Time Complexity: O(logN), where N is the number of elements.

```
v.push_back(new_value);   // add to end
push_heap(v.begin(), v.end());  // restore max-heap
```

```
// C++ program to demonstrate the use of push_heap()
// function
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

void print(vector<int>& vc)
{
    for (auto i : vc) {
        cout << i << " ";
    }
    cout << endl;
}

int main()
{
    vector<int> vc{ 20, 30, 40, 10 };

    make_heap(vc.begin(), vc.end());
    cout << "Initial Heap: ";
    print(vc);

    vc.push_back(50);
    cout << "Heap just after push_back(): ";
    print(vc);
    push_heap(vc.begin(), vc.end());
    cout << "Heap after push_heap(): ";
    print(vc);

    return 0;
}
```

Pop heap (max heap)

- Move largest element to back of vector
- So we can pop_back() vector
- Use custom comparator for min heap

Time Complexity: O(logN), where N is the number of elements.

```
pop_heap(v.begin(), v.end());  // move max to end
v.pop_back();                  // actually remove it
```

pop_heap moves swap the first with the last,
and restore the heap from [begin, end - 1)

```
int main()
{
    // initial vector
    vector<int> vc{ 40, 10, 20, 50, 30 };

    // making heap
    make_heap(vc.begin(), vc.end());
    cout << "Initial Heap: ";
    print(vc);

    // using pop_heap() function to move the largest element
    // to the end
    pop_heap(vc.begin(), vc.end());
    cout << "Heap after pop_heap(): ";
    print(vc);

    // actually removing the element from the heap using
    // pop_back()
    vc.pop_back();
    cout << "Heap after pop_back(): ";
    print(vc);

    return 0;
}
```

Priority Queue STL

Queue library

Building min-heap

Add item to our heap

Peek top

Remove top

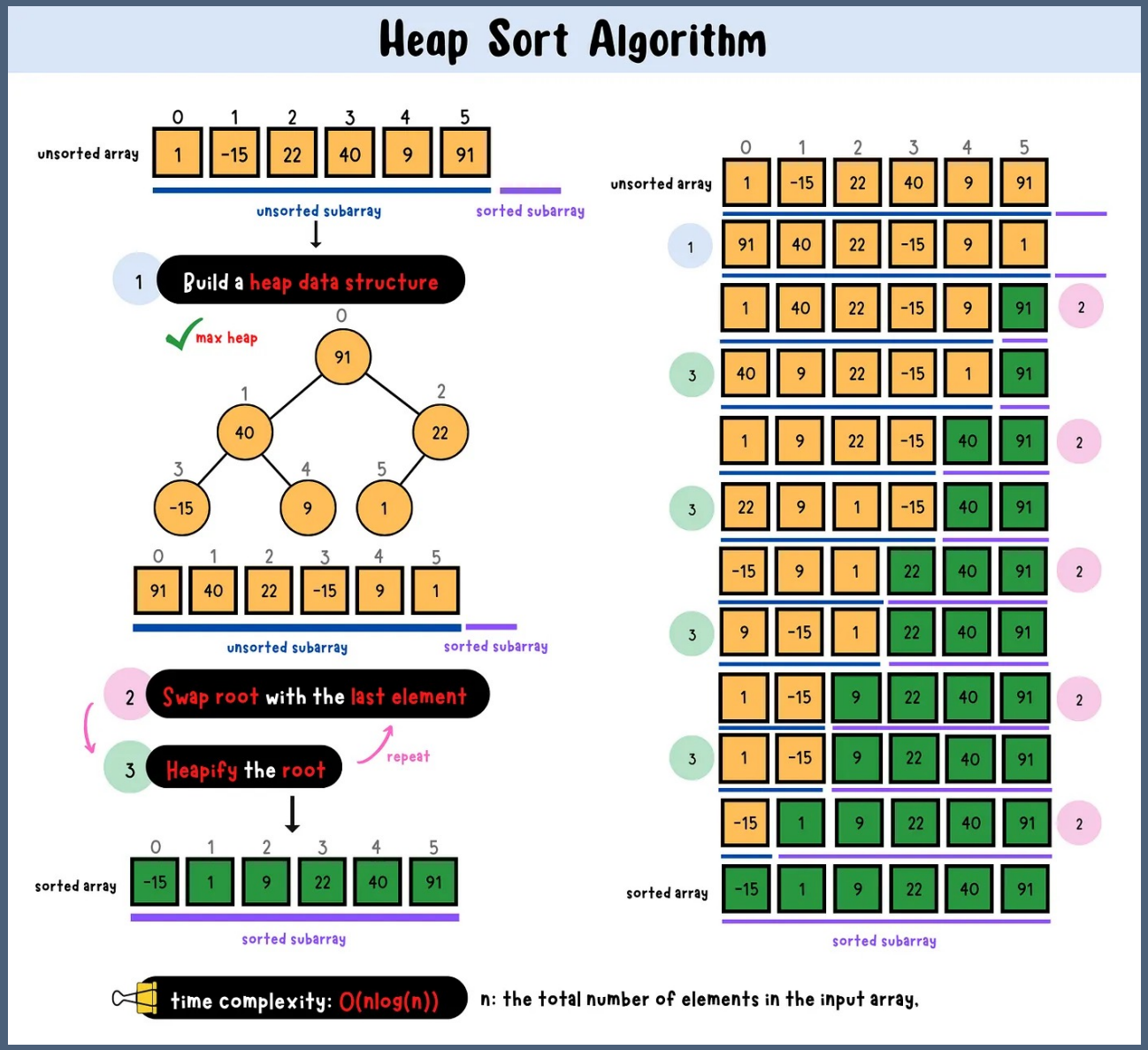
```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    priority_queue<int, vector<int>, greater<int>> pq; // min-heap
    pq.push(19);
    pq.push(35);
    pq.push(11);

    cout << "Top: " << pq.top() << endl; // smallest

    pq.pop(); // remove top

    cout << "New Top: " << pq.top() << endl;
    return 0;
}
```

So, for a priority queue

We can use Heap for this

How do we sort a list with heap?

Yes, that's called "Heap Sort"



- Build a **max-heap** from the unsorted array
- Swap **root (max)** with the last element
- Reduce heap size (ignore last element)
- **Heapify down from the root**
- Repeat until fully sorted

https://yuminlee2.medium.com/heap-sort-algorithm-6e200dc51845

Heapsort is not stable

“Heap sort is not stable: equal elements may change order due to swaps”

Let’s Check Your Understanding

#1

กำหนด array ของตัวเลขดังต่อไปนี้:

[6, 12, 3, 11, 14, 8, 1, 7, 15, 9, 2]

จงหา **min-heap** ที่ได้จากการเรียกใช้อัลกอริทึม **make-heap (build-heap)**

#2

ในอัลกอริทึม **MERGE-SORT** เราจะรวม (merge) ลิสต์ที่เรียงลำดับแล้ว 2 ลิสต์ ให้เป็นลิสต์ที่เรียงลำดับแล้วลิสต์เดียวในเวลา **O(n)**

จงอธิบายอัลกอริทึมที่ใช้เวลา **O(n log k)** ในการรวมลิสต์ที่เรียงลำดับแล้วจำนวน **k ลิสต์** ให้เป็นลิสต์เดียว

โดยที่ **n** คือจำนวนสมาชิกทั้งหมดในทุกลิสต์อินพุต
และให้อธิบายด้วยว่าเหตุใดอัลกอริทึมของคุณจึงทำงานในเวลา **O(n log k)**

#3

กำหนด array ที่มีสมาชิกจำนวน **n ตัว** โดยที่แต่ละตัวจะอยู่ห่างจากตำแหน่งที่ถูกต้องในลิสต์ที่เรียงแล้วไม่เกิน **k ตำแหน่ง**
จงออกแบบอัลกอริทึมเพื่อเรียงลำดับ array นี้ให้เสร็จในเวลา **O(n log k)**

ตัวอย่างเช่น ถ้า **k = 2**
สมาชิกที่ควรอยู่ที่ตำแหน่งที่ 7 ใน array ที่เรียงแล้ว อาจปรากฏอยู่ในตำแหน่ง **5, 6, 7, 8, 9** ในอาร์เรย์ ณ ปัจจุบัน

ตัวอย่างที่ 1
Input:
arr[] = {6, 5, 3, 2, 8, 10, 9}, k = 3
Output:
arr[] = {2, 3, 5, 6, 8, 9, 10}

ตัวอย่างที่ 2
Input:
arr[] = {10, 9, 8, 7, 4, 70, 60, 50}, k = 4
Output:
arr[] = {4, 7, 8, 9, 10, 50, 60, 70}

#4

กำหนด vector ชื่อ stones ที่เก็บน้ำหนักของก้อนหินหลายก้อน ซึ่งยังไม่ได้เรียงลำดับ
ในแต่ละรอบ ให้นำก้อนหินที่มีน้ำหนักมากที่สุด 2 ก้อนมาชนกัน
ถ้าน้ำหนักเท่ากัน ก้อนหินทั้งสองจะถูกทำลาย
ถ้าน้ำหนักไม่เท่ากัน ก้อนที่เบากว่าจะถูกทำลาย และก้อนที่หนักกว่าจะเหลือน้ำหนักเป็นผลต่างของทั้งสองก้อน
จงเขียนฟังก์ชัน `int lastStoneWeight(vector<int>& stones);` ที่นำเข้า vector stones แล้วคืนค่าน้ำหนักของก้อนหินก้อนสุดท้าย หรือคืนค่า 0 หากไม่เหลือหินเลย

```
stones = {2, 7, 4, 1, 8, 1}
```